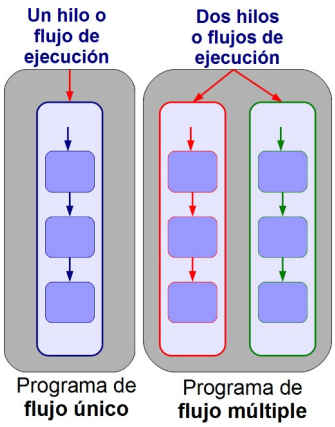
Programación de Servicios y Procesos

U2: Programación Multihilo

1. Introducción

Seguro que en más de una ocasión mientras te descargabas una imagen desde tu navegador web, seguías navegando por Internet e incluso iniciabas la descarga de un nuevo archivo, y todo esto ejecutándose el navegador como un único proceso, es decir, teniendo un único ejemplar del programa en ejecución.



Pues bien, ¿Cómo es capaz de hacer el navegador web varias tareas a la vez? Seguro que estarás pensando en la **programación concurrente**, y así es, pero con un nuevo enfoque de la concurrencia, denominado "**programación multihilo**". Justo lo que vamos a estudiar en esta unidad.

Los programas realizan actividades o tareas, y para ello pueden seguir uno o más flujos de ejecución. Dependiendo del número de flujos de ejecución, podemos hablar de dos tipos de programas:

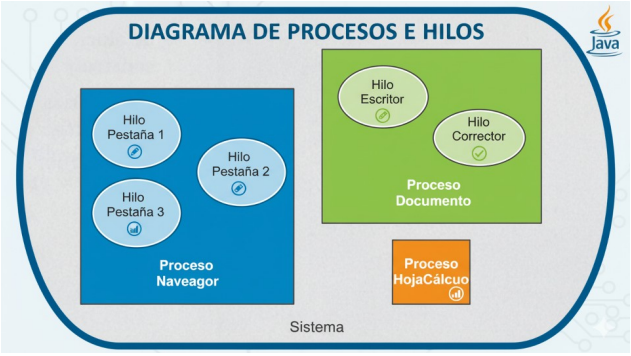
- **Programa de flujo único.** Es aquel que realiza las actividades o tareas que lleva a cabo una a continuación de la otra, de manera **secuencial**, lo que significa que cada una de ellas debe concluir por completo, antes de que pueda iniciarse la siguiente.

- **Programa de flujo múltiple.** Es aquel que coloca las actividades a realizar en diferentes flujos de ejecución, de manera que cada uno de ellos se inicia y termina por separado, pudiéndose ejecutar éstos de manera concurrente.

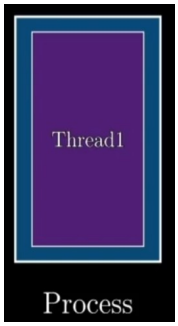
La **programación multihilo** o **multithrearing** consiste en desarrollar programas o aplicaciones de flujo múltiple. Cada uno de esos flujos de ejecución es un **thread** o **hilo**.

En el ejemplo anterior sobre el navegador web, un hilo se encargaría de la descarga de la imagen, otro de continuar navegando y otro de iniciar una nueva descarga. La utilidad de la programación multihilo resulta evidente en este tipo de aplicaciones. El navegador puede realizar "a la vez" estas tareas, por lo que no habrá que esperar a que finalice una descarga para comenzar otra o seguir navegando.

Cuando decimos "a la vez" recuerda que nos referimos a que las tareas se realizan concurrentemente, pues el que las tareas se ejecuten realmente en paralelo dependerá del Sistema Operativo y del número de procesadores del sistema donde se ejecute la aplicación. En realidad, esto es transparente para el programador y usuario, lo importante es la sensación real de que el programa realiza de forma simultánea diferentes tareas.



2. Conceptos sobre hilos

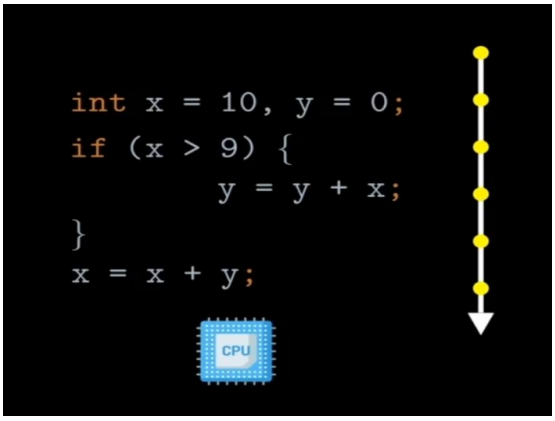


Pero ¿qué es realmente un hilo? Un **hilo** es un flujo de control secuencial independiente dentro de un proceso y está asociado con una secuencia de instrucciones, un conjunto de registros y una pila.

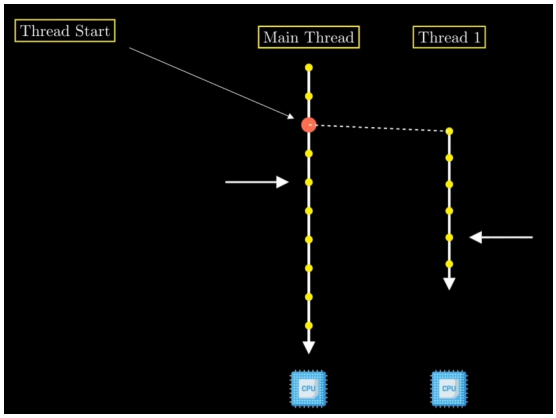
Cuando se ejecuta un programa, el Sistema Operativo crea un proceso y también crea su primer hilo, **hilo primario**, el cual puede a su vez crear hilos adicionales.

Desde este punto de vista, un proceso no se ejecuta, sino que solo es el espacio de direcciones donde reside el código que es ejecutado mediante uno o más hilos.

Flujo único

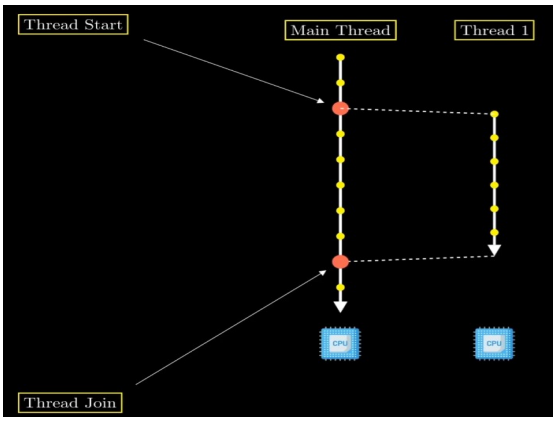


Flujo múltiple: Dos hilos independientes

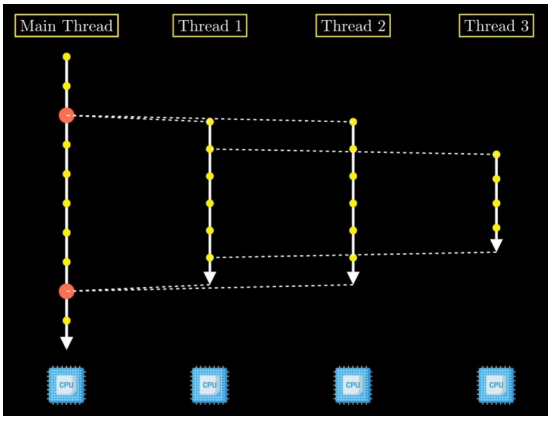


Cuando un programa Java se lanza (se convierte en un proceso) empieza a ejecutarse por su método ***main()*** que lo ejecuta el thread principal (**Main Thread**), un hilo especial creado por la JVM para ejecutar la aplicación.

Flujo múltiple: Dos hilos dependientes



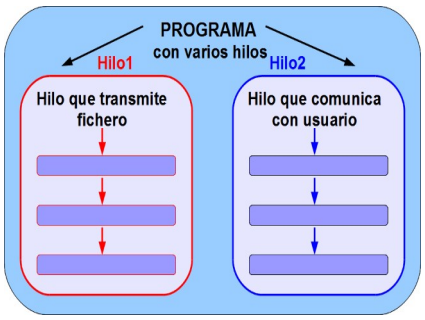
Flujo múltiple: Cuatro hilos dependientes



Desde un proceso se pueden crear e iniciar tantos threads como sean necesarios. Estos hilos ejecutarán partes del código de la aplicación de forma concurrente con el thread principal.

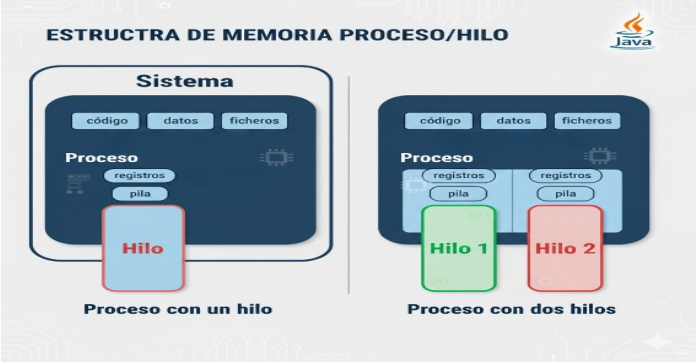
Por lo tanto podemos hacer las siguientes **observaciones**:

- Un hilo no puede existir independientemente de un proceso. Por tanto, si el proceso finaliza por alguna circunstancia, todos sus hilos también lo hacen.
- Un hilo no puede ejecutarse por si solo.
- Dentro de cada proceso puede haber varios hilos ejecutándose.



Un único hilo es similar a un programa secuencial, por si mismo no nos ofrece nada nuevo. Es la habilidad de ejecutar varios hilos dentro de un proceso lo que ofrece algo nuevo y útil ya que cada uno de estos hilos puede ejecutar actividades diferentes al mismo tiempo. Así en un programa un hilo puede encargarse de la comunicación con el usuario, mientras que otro hilo transmite un fichero, otro puede acceder a recursos del sistema (cargar sonidos, leer ficheros, …), etc.

2.1. Recursos compartidos por los hilos

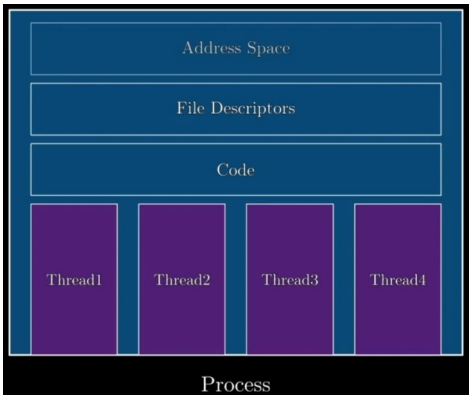


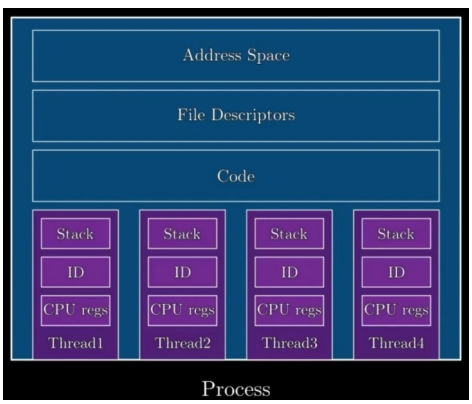
Un hilo lleva asociados los siguientes elementos:

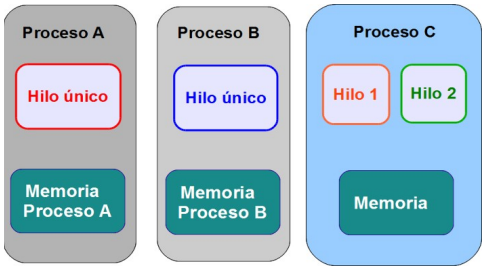
- Un identificador único que lo identifica dentro del proceso.
- Un contador de programa propio.
- Un conjunto de registros de la CPU.
- Una pila (usada para almacenar variables locales y parámetros de métodos).

Por otra parte, **un hilo puede compartir con otros hilos del mismo proceso los siguientes recursos:**

- Código.
- Espacio de direcciones usado por el proceso.
- Datos (como variables globales).
- Otros recursos del sistema operativo, como los ficheros abiertos.

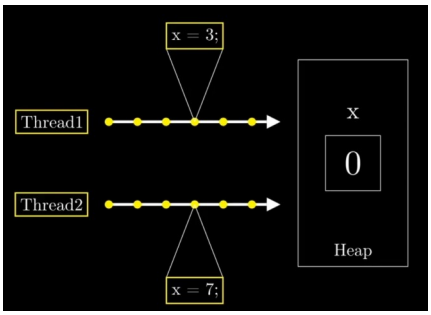






Seguro que te estarás preguntando "si los hilos de un proceso comparten el mismo espacio de memoria, ¿qué pasa si uno de ellos la corrompe?" La respuesta es que los otros hilos también sufrirán las consecuencias. Recuerda que en el caso de procesos, el sistema operativo normalmente protege a un proceso de otro y si un proceso corrompe su espacio de memoria los demás no se verán afectados

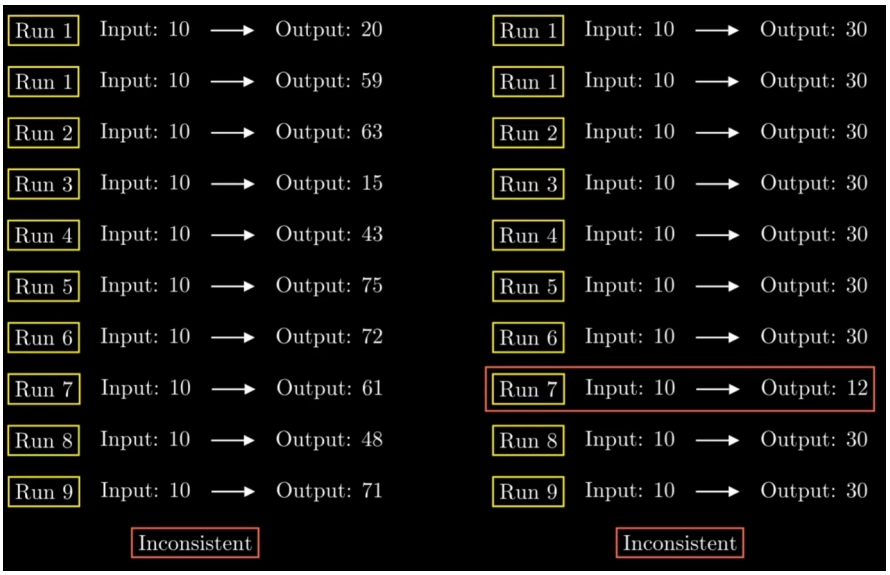
El hecho de que los hilos compartan recursos (por ejemplo, pudiendo acceder a las mismas variables) implica que sea necesario **utilizar esquemas de bloqueo y sincronización,** lo que puede hacer más difícil el desarrollo de los programas y así como su depuración.



Por ejemplo, si dos hilos quieren, en el mismo instante de tiempo, actualizar con diferentes valores una variable x compartida, es necesario garantizar el orden de la operación para tener un **comportamiento consistente**. De lo contrario se tendría un resultado impredecible, algo que debe evitarse cuando se implementan aplicaciones multihilo ya que provocará que se tengan diferentes salidas en diferentes ejecuciones.

COMPORTAMIENTO INCONSISTENTE

COMPORTAMIENTO INDETERMINISTA

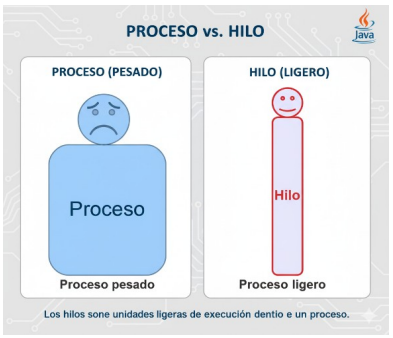


Realmente, es en la sincronización de hilos donde reside el arte de programar con hilos ya que de no hacerlo bien, podemos crear una aplicación totalmente ineficiente o inútil, como por ejemplo, programas que tardan horas en procesar servicios, que se bloquean con facilidad o que intercambian datos de manera equivocada.

Profundizaremos más adelante en la sincronización, comunicación y compartición de recursos entre hilos dentro del contexto de Java.

Pero ¿qué ventajas aportan los hilos y cuando deben utilizarse?

2.2. Ventajas y usos de hilos



Como consecuencia de compartir el espacio de memoria, los hilos aportan las siguientes **ventajas sobre los procesos**:

- Se consumen menos recursos en el lanzamiento y la ejecución de un hilo que en el lanzamiento y ejecución de un proceso.
- Se tarda menos tiempo en crear y terminar un hilo que un proceso.
- La conmutación entre hilos del mismo proceso o **cambio de contexto** es bastante más rápida que entre procesos.

Es por esas razones, por lo que a los hilos se les denomina también **procesos ligeros**.

Y ¿**cuándo se aconseja utilizar hilos**? Se aconseja utilizar hilos en una aplicación cuando:

- La aplicación maneja entradas de varios dispositivos de comunicación.
- La aplicación debe poder realizar diferentes tareas a la vez.
- Interesa diferenciar tareas con una prioridad variada. Por ejemplo, una prioridad alta para manejar tareas de tiempo crítico y una prioridad baja para otras tareas.
- La aplicación se va a ejecutar en un entorno multiprocesador.

Por ejemplo, imagina la siguiente situación:

Debes crear una aplicación que se ejecutará en un servidor para atender peticiones de clientes. Esta aplicación podría ser un servidor de bases de datos, o un servidor web.

- Cuando se ejecuta el programa éste abre su puerto y queda a la escucha, esperando recibir peticiones.
- Si cuando recibe una petición de un cliente se pone a procesarla para obtener una respuesta y devolverla, cualquier petición que reciba mientras tanto no podrá atenderla, puesto que está ocupado.

La solución será construir la aplicación con múltiples hilos de ejecución.

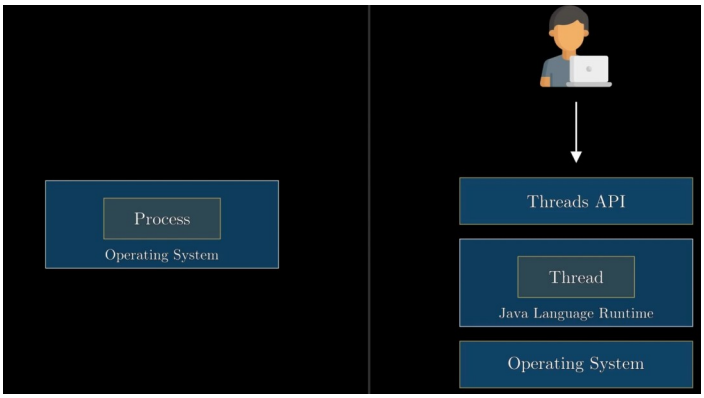
- En este caso, al ejecutar la aplicación se pone en marcha el hilo principal, que queda a la escucha.
- Cuando el hilo principal recibe una petición, creará un nuevo hilo que se encarga de procesarla y generar la consulta, mientras tanto el hilo principal sigue a la escucha recibiendo peticiones y creando hilos.
- De esta manera un gestor de bases de datos puede atender consultas de varios clientes, o un servidor web puede atender a miles de clientes.
- Si el número de peticiones simultáneas es elevado, la creación de un hilo para cada una de ellas puede comprometer los recursos del sistema. En este caso, lo resolveremos mejor con un pool de hilos (contenedor de hilos).

Resumiendo, los hilos son idóneos para programar aplicaciones de entornos interactivos y en red, así como simuladores y animaciones.

2.3. Threads Vs Procesos

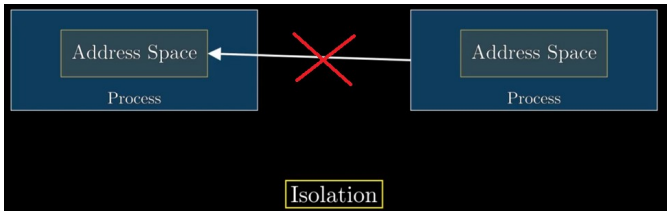
Primera diferencia: Quién los gestiona

Los procesos son objetos con los que se trabaja a nivel de sistema operativo. En cambio, las hebras son objetos con los que se trabaja a nivel de ***Java Runtime*** (se ejecuta un nivel por encima del Sistema Operativo). Por tanto, será el ***Java Runtime*** el encargado de gestionar los hilos y todas sus complejidades.



Segunda diferencia: Aislamiento.

Cada proceso está asociado a una aplicación en ejecución, teniendo cada uno de ellos su propio espacio de direcciones al cual no va a poder acceder ningún otro proceso (**aislamiento**).



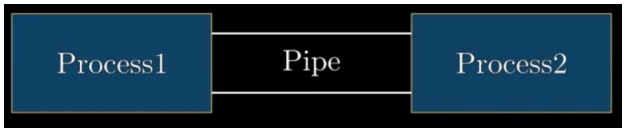
Tercera diferencia: La comunicación.

¿Cómo pueden comunicarse los procesos? Como se ha visto en la unidad anterior, si ambos procesos están ejecutándose en la misma máquina, una de las formas más fáciles de lograrlo es mediante el uso de ficheros. Por ejemplo, el proceso 1 produce unos datos que necesita pasar el proceso 2. Para ello los escribe en un fichero y el proceso 2, los lee desde fichero.

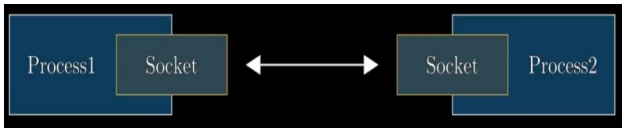


En esta solución, ambos procesos necesitan saber el nombre del fichero, tener los permisos correspondiente y acceso exclusivo para que uno no intente leer antes de que el otro haya escrito.

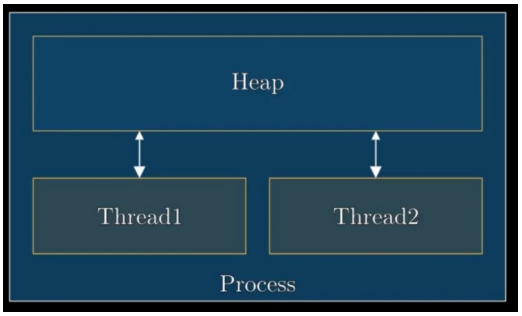
También se puede usar el concepto de Pipe visto en la unidad anterior.



En cambio, si los dos procesos están en máquinas diferentes se puede recurrir al uso de sockets que serán vistos en la unidad siguiente.



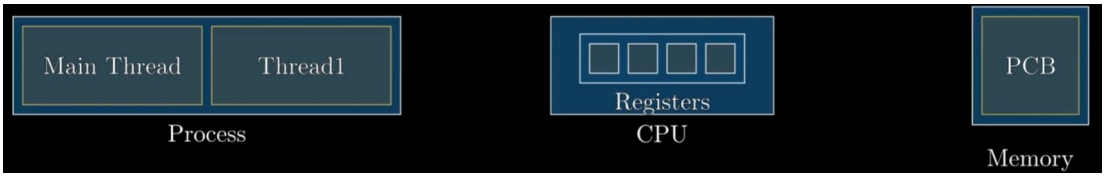
A la hora de llevar a cabo una **comunicación entre hilos**, esta se puede llevar a cabo de un modo muy simple debido a que los hilos pueden compartir el espacio de direcciones del proceso que los crea:



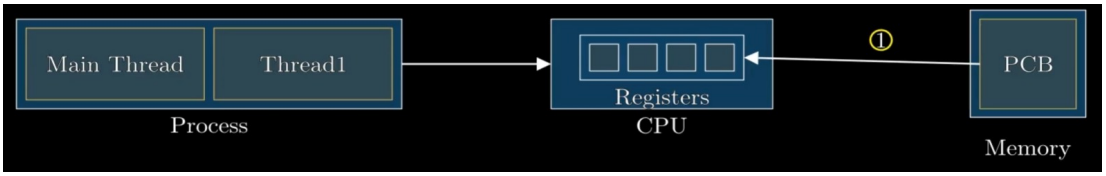
Por tanto, no son necesarios métodos sofisticados ni complejos para efectuar el intercambio de datos ya que los distintos hilos de un mismo proceso comparten la memoria asignada al proceso. Sin embargo, los **hilos deben coordinarse para el acceso a los contenidos de la memoria y a los ficheros, lo cual hace que esa coordinación y sincronización sea la parte complicada de uso**.

Cuarta diferencia: El cambio de contexto entre procesos es más costoso que el cambio de contexto entre hilos.

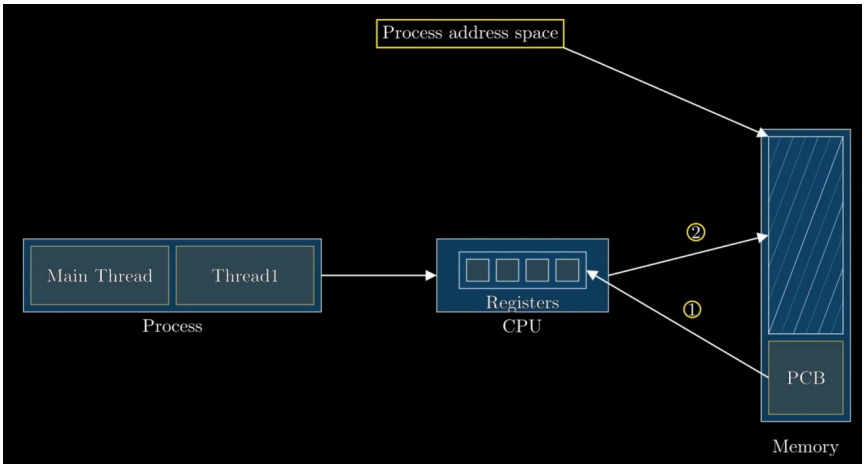
Partimos de una situación en la que se tiene un proceso con su hilo principal y que crea un nuevo hilo.



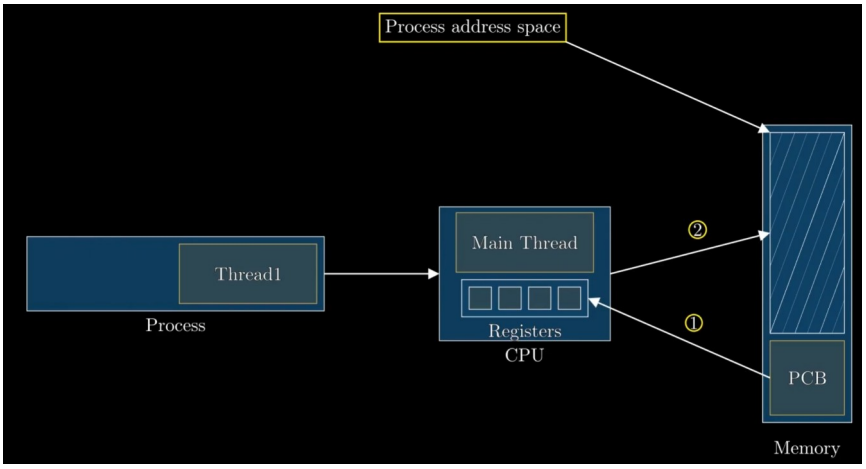
Este proceso está activo y por tanto, cuando el scheduler del sistema operativo le asigna la CPU, se cargan en los registros de la CPU toda la información necesaria leída del PCB.



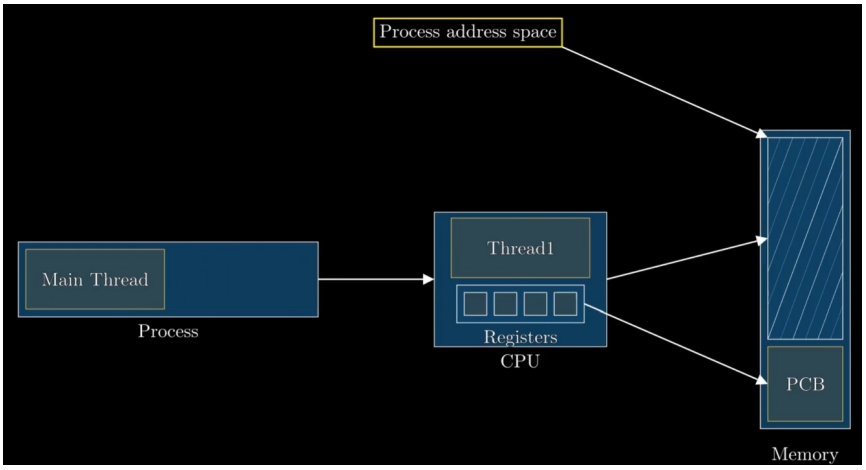
Además, se reserva espacio en memoria para cargar todo el espacio de direcciones del proceso.



A continuación el hilo principal comienza su ejecución.



Cuando la CPU vaya a ejecutar el hilo Thread1, el sistema operativo debe actualizar el PCB del proceso, sacar del procesador al hilo principal y asignar el procesador al hilo Thread1.

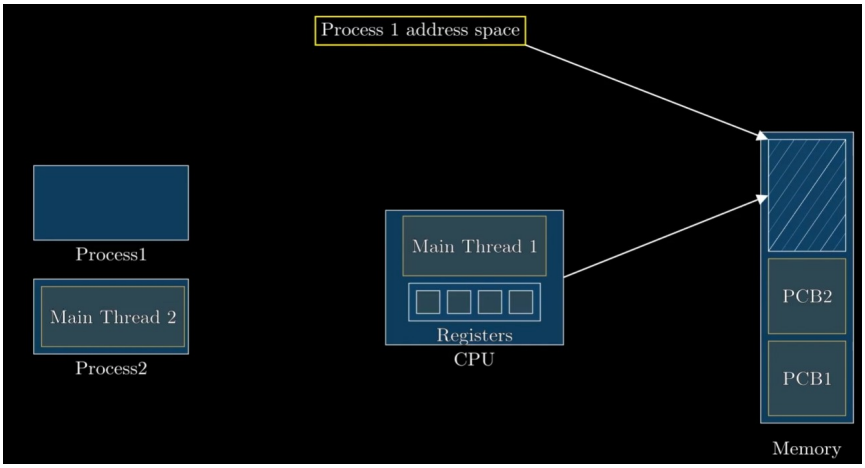


Esto que se acaba de realizar, es lo que se conoce como **cambio de contexto del hilo**.

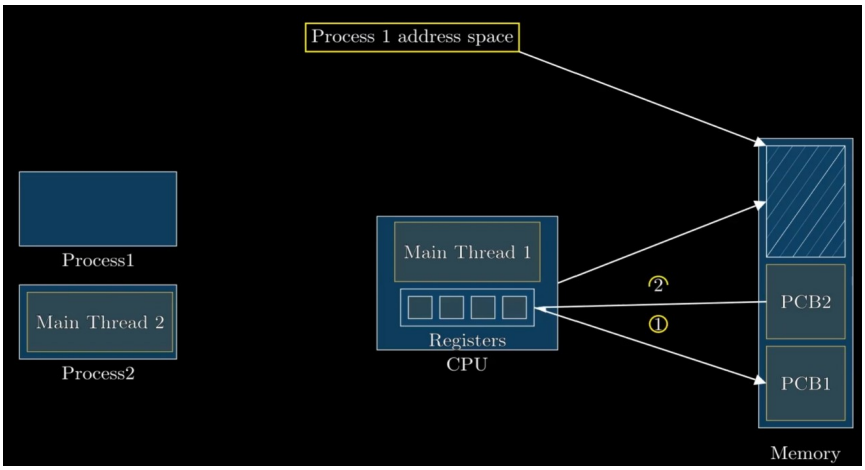
Supongamos ahora que, en lugar de un proceso con dos hilos, tenemos dos procesos con sus respectivos hilos principales. Ambos procesos están activos, y por consiguiente, a cualquiera de los dos se le podrá asignar la CPU.



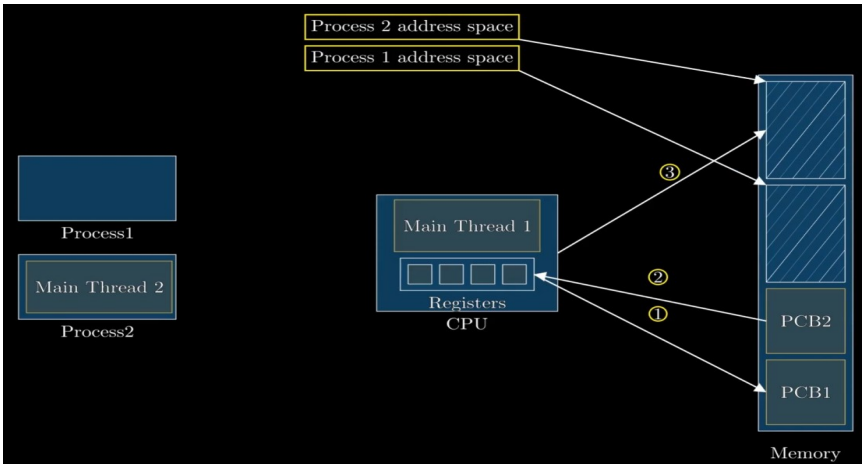
El scheduler del sistema operativo decide asignar la CPU al primero de los procesos:



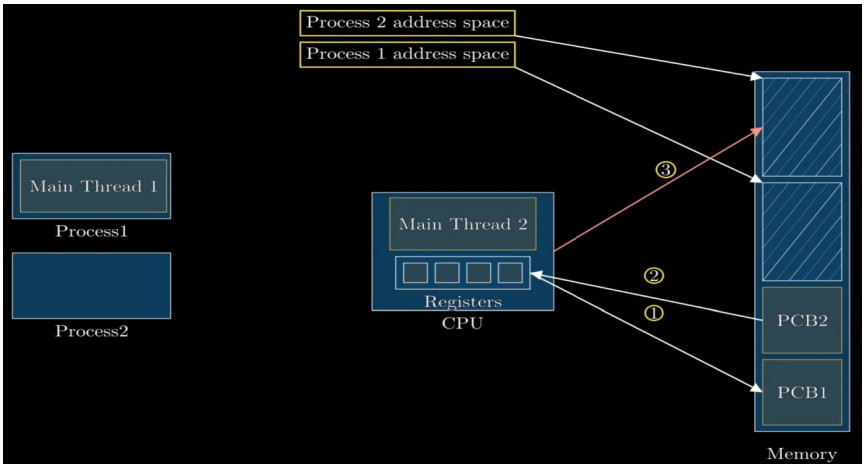
Si el scheduler le arrebata la CPU al proceso 1 para asignarla al proceso 2, debe haber un cambio de contexto a nivel de proceso. Por tal motivo, debe guardar todo el contexto del proceso 1 en su PCB (PCB1). También deberá cargar en los registros de la CPU toda la información necesaria del proceso 2, leyéndola del PCB2.



Además, necesita cargar en memoria el espacio de direcciones del proceso 2:



Finalmente, el hilo principal del proceso 2 comenzará/continuará con su ejecución:



Se puede observar que el cambio de contexto entre procesos, supone una sobrecarga en comparación con el cambio de contexto entre hilos.

3. Multihilos en Java. Librería y clases



Java da soporte al concepto de hilo desde el propio lenguaje, con algunas **clases e interfaces** definidas en el paquete java.lang y con **métodos** específicos para la manipulación de hilos en la clase Object. A partir de la versión Java 5.0, se incluye el paquete java.util.concurrent con nuevas utilidades para desarrollar aplicaciones multihilo e incluso aplicaciones con un alto nivel de concurrencia.

3.1. Utilidades de concurrencia del paquete java.lang

Dentro del **paquete** java.lang disponemos de una interfaz y las siguientes clases para trabajar con hilos:

- **Clase** Thread. Es la clase responsable de producir hilos funcionales para otras clases y proporciona gran parte de los métodos utilizados para su gestión.

  Tiene algunos métodos estáticos importantes, entre los que se pueden destacar:

  - currentThread(): Nos devuelve un objeto de tipo Thread a través del cual se puede acceder a toda la información sobre la hebra que está actualmente ejecutando el código.
  - sleep(): Provoca que el hilo se duerma (se suspenda) una determinada cantidad de tiempo en milisegundos. Necesita tratar la excepción InterruptedException.
  - getName(): Nos devuelve el nombre del hilo. Por defecto, el nombre del hilo principal se llama ***main***. En caso de no asignarle ningún nombre, a los hilos que se vayan creando se les asignará un nombre por defecto con el formato Thread-xx, siendo xx un número que empieza en 0.
  - setName(): Permite cambiar el nombre del hilo.
  - start(): Método que inicia o arranca el hilo.

Paquete basicos: MetodosEstaticosThreads

- **Interfaz** Runnable. Proporciona la capacidad de añadir la funcionalidad de hilo a una clase simplemente implementando la interfaz, en lugar de derivándola de la clase Thread.
- **Clase** ThreadDeath**.** Es una clase de error, deriva de la clase Error, y proporciona medios para manejar y notificar errores.
- **Clase** ThreadGroup**.** Esta clase se utiliza para manejar un grupo de hilos de modo conjunto, de manera que se pueda controlar su ejecución de forma eficiente.
- **Clase** Object. Esta clase no es estrictamente de apoyo a los hilos, pero proporciona unos cuantos métodos cruciales dentro de la arquitectura multihilo de Java. Estos métodos son wait(), notify() y notifyAll().

3.2. Kernel Threads Vs Java Threads

El **Kernel Thread** es la unidad básica de ejecución que el sistema operativo puede asignar a los cores de la CPU y es gestionado directamente por el kernel del sistema operativo.

Por tanto, si un ordenador tiene 8 núcleos, el kernel puede distribuir 8 Kernel Threads diferentes para que se ejecuten al mismo tiempo de manera 100% simultánea.

En cambio, los **Java Threads (User Threads)** son los hilos creados desde código. El sistema operativo no sabe que existen (para el kernel, un programa es solo un gran proceso). Por tanto, será el propio lenguaje de programación (en este caso la JVM) quien se encargue de administrar los Java Threads.

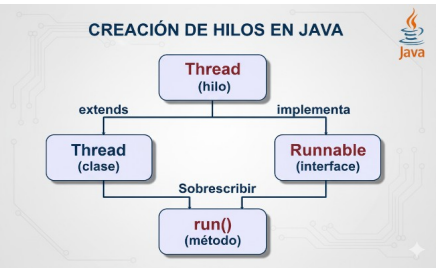
Tradicionalmente cada Java Thread se mapea 1:1 con un Kernel Thread, es decir, la JVM crea el Java Thread y se lo asigna al sistema operativo para que este lo mapee a un Kernel Thread donde será ejecutado.

4. Creación de hilos

En Java, un **hilo** se representa mediante una instancia de la clase java.lang.thread. Este **objeto thread** se emplea para iniciar, detener o cancelar la ejecución del hilo de ejecución.

Los hilos o threads se pueden **implementar o definir de dos formas**:

- Extendiendo la clase Thread.
- Implementando la interfaz Runnable.



En **ambos casos, se debe proporcionar una definición del método** *run(),* ya que este método es el que contiene el código que ejecutará el hilo, es decir, su comportamiento.

El procedimiento de construcción de un hilo es independiente de su uso, pues una vez creado se emplea de la misma forma.

Constructores	
Thread()	Crea un hilo que hereda de Thread y su nombre es asignado por Java.
Thread(Runnable target)	Crea un hilo que implementa la interfaz Runnable y su nombre es asignado por Java.
Thread(String name)	Crea un hilo que hereda de Thread y cuyo nombre es name.
Thread(Runnable target, String name)	Crea un hilo que implementa la interfaz Runnable y cuyo nombre es name

Entonces, ¿**cuando utilizar uno u otro procedimiento?** No hay nada que indique que una forma es mejor que otra. Ambos métodos son similares y el resultado es el mismo.

- Extender la clase Thread es el procedimiento más sencillo, pero no siempre es posible. Si la clase ya hereda de alguna otra clase padre, no será posible heredar también de la clase Thread (recuerda que Java no permite la herencia múltiple), por lo que habrá que recurrir al otro procedimiento.
- El método preferido debería ser implementar Runnable, y pasarle la instancia al constructor de Thread. Implementar Runnable siempre es posible, es el procedimiento más general y también el más flexible.

RECUERDA: Cuando la Máquina Virtual Java (JVM) arranca la ejecución de un programa, ya hay un hilo ejecutándose, denominado **hilo principal del programa**, controlado por el método main(), que se ejecuta cuando comienza el programa y es el último hilo que termina su ejecución, ya que cuando este hilo finaliza, el programa termina.

4.1. Creación de hilos extendiendo de la clase Thread

Para **definir** y **crear un hilo extendiendo la clase Thread**, haremos lo siguiente:

- Implementar una nueva clase que herede de la clase Thread.
- Redefinir en la nueva clase el método run() con el código asociado al hilo. Son las sentencias que ejecutará el hilo.
- Declarar y crear un objeto de la nueva clase Thread. Éste será realmente el hilo.

Una vez creado el hilo, para **ponerlo en marcha o iniciarlo**:

- Invocar al método start() del objeto thread (el hilo que hemos creado).

Paquete basicos.herencia: CreacionHilosHeredando; MaximoArray

4.2. Creación de hilos implementado la interfaz Runnable

Para **definir y crear hilos implementando la interfaz Runnable** seguiremos los siguientes pasos:

- Crear una nueva clase que implemente Runnable.
- Redefinir en la nueva clase el método run() con el código asociado al hilo. Son las sentencias que ejecutará el hijo
- Declarar un objeto de la clase Thread y se llama al constructor de la clase Thread pasando al constructor un objeto cuya clase implementa Runnable. Este será realmente el hilo.

Una vez creado el hilo, para ponerlo en marcha o iniciarlo:

- Invocar al método start() del objeto thread (el hilo que hemos creado).

Paquete basicos.runnable: CreacionHilosRunnable; CoheteRunnable

4.3. Creación de hilos mediante clases anónimas que implementan la interfaz Runnable

Para ello seguiremos los siguientes pasos:

- Declarar y crear un objeto de tipo Runnable.
- Redefinir el método run() con el código asociado al hilo.
- Declarar un objeto de la clase Thread y se instancia llamando al constructor de la clase Thread pasando al constructor, el objeto runnable previamente declarado y creado.

Una vez creado el hilo, para ponerlo en marcha o iniciarlo:

- Invocar al método start() del objeto thread (el hilo que hemos creado).

Paquete basicos.anonima: CreacionHilosAnonimoRunnable

4.4. Creación de hilos mediante expresiones lambda

Permite hacer lo mismo que en el caso anterior pero de una forma más elegante. Para ello seguiremos los siguientes pasos:

- Declarar un objeto de tipo Runnable y se le asocia la expresión lambda que incluye el código asociado al hilo.
- Declarar un objeto de la clase Thread y se instancia llamando al constructor de la clase Thread pasando al constructor, el objeto runnable previamente declarado y creado.

Una vez creado el hilo, para ponerlo en marcha o iniciarlo:

- Invocar al método start() del objeto thread (el hilo que hemos creado).

Esto tiene sentido usarlo en el caso de que la tarea la vamos a usar en un solo punto de la aplicación y no se va a volver a instanciar.

Paquete basicos.lambda: CreacionHilosExpresionLambda; MainPayment y Payment; Cohete y SimulacionCoheteExpresionLambda

4.5. Iniciar un hilo



Cuando se crea un nuevo hilo mediante el método new(), esto no implica que el hilo ya se pueda ejecutar.

Para que el hilo se pueda ejecutar, debe estar en el estado “Ejecutable”, y para

conseguir ese estado es necesario **iniciar o arrancar el hilo** mediante el método start() de la clase Thread.

En realidad el método start() **realiza las siguientes tareas:**

- Crea los recursos del sistema necesarios para ejecutar el hilo.
- **Llama a su método** run().

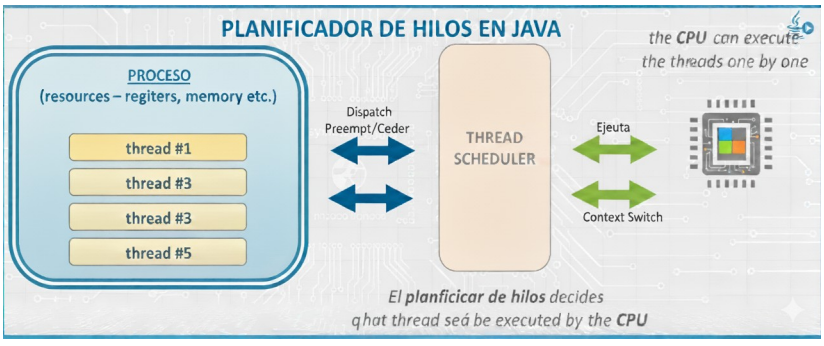
Es por esto último que cuando se invoca a start() se suele decir que el hilo está **Runnable**, pero esto no significa que el hilo esté ejecutándose en todo momento, ya que **un hilo "Ejecutable" puede estar "Runnable" o "Running"** según tenga o no asignado tiempo de procesamiento.

Algunas **consideraciones importantes** que debes tener en cuenta son las siguientes:

- Puedes invocar directamente al método run(), por ejemplo poner hilo1.run(); y se ejecutará el código asociado a run() dentro del hilo actual (como cualquier otro método), pero no comenzará un nuevo hilo.
- Una vez que se ha llamado al método start() de un hilo, no puedes volver a realizar otra llamada al mismo método. Si lo haces, obtendrás una excepción IllegalThreadStateException.
- **Orden de ejecución no-determinístico**: El orden en el que se inician los hilos mediante start() no influye en el orden de ejecución de los mismos (no se conoce la secuencia en la que serán ejecutadas la instrucciones del programa).

5. Prioridades de hilos

En Java, **cada hilo tiene una prioridad** representada por un valor de tipo entero **entre 1 y 10**. Cuanto mayor es el valor, mayor es la prioridad del hilo.



Por defecto, el **hilo principal** de cualquier programa, o sea, el que ejecuta su método main() siempre es creado con prioridad 5.

El resto de **hilos secundarios** (creados desde el hilo principal, o desde cualquier otro hilo en funcionamiento), **heredan la prioridad que tenga en ese momento su hilo padre**.

En la clase Thread se definen 3 constantes para manejar estas prioridades:

- MAX_PRIORITY (= 10). Es el valor que simboliza la máxima prioridad.
- MIN_PRIORITY (=1). Es el valor que simboliza la mínima prioridad.
- NORM_PRIORITY (= 5). Es el valor que simboliza la prioridad normal, la que tiene por defecto el hilo donde corre el método main().

Además en cualquier momento se puede **obtener** y **modificar la prioridad de un hilo**, mediante los siguientes métodos de la clase Thread:

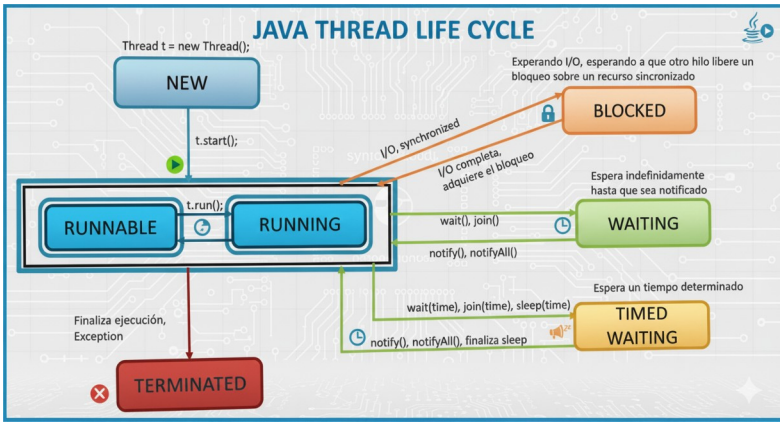
- getPriority(). Obtiene la prioridad de un hilo. Este método devuelve la prioridad del hilo.
- setPriority(int prioridad). Modifica la prioridad de un hilo. Este método toma como argumento un entero entre 1 y 10, que indica la nueva prioridad del hilo.

Por tanto, Java tiene 10 niveles de prioridad que no tienen por qué coincidir con los del sistema operativo sobre el que está corriendo. Por ello, lo mejor es que utilices en tu código sólo las constantes MAX_PRIORITY, NORM_PRIORITY y MIN_PRIORITY.

Podemos conseguir **aumentar el rendimiento de una aplicación multihilo** gestionando adecuadamente las prioridades de los diferentes hilos, por ejemplo utilizando una prioridad alta para tareas de tiempo crítico y una prioridad baja para otras tareas menos importantes.

Paquete prioridades: SaludosDesdeVariosHilos; CreacionVariosHilos; PrioridadesHilos

6. Estados de un hilo



El **ciclo de vida de un hilo** comprende los diferentes estados en los que puede estar desde que se crea hasta que finaliza.

- **NEW**

  Estado de un hilo cuando se crea con el operador *new*. En este estado el hilo aún no se ha iniciado, es decir, no se ha comenzado a ejecutar el código del método *run()* del hilo.

- **RUNNABLE**

  Estado de un hilo cuando está listo para ejecutarse, es decir, está esperando a que se le asigne la CPU. Pasará al estado de RUNNING cuando el scheduler de hilos lo selecciona y lo asigna a uno de los núcleos de la CPU, pasando a ejecutar las sentencias de su método *run()*.

- **BLOCKED**

  En este estado el hilo está bloqueado esperando a que otro hilo libere un recurso por el que compiten. También se encuentra en este estado aquellos hilos que están esperando a que se complete una operación de E/S.

- **WAITING**

  En este estado el hilo espera indefinidamente hasta que otro hilo realice una acción concreta. Pasará al estado de RUNNABLE cuando otro hilo realice una notificación (*notify*).

- **TIMED_WAITING**

  En este estado se encuentran aquellos hilos que están esperando un tiempo específico a que otro hilo realice una acción. Pasará al estado RUNNABLE cuando otro hilo realice una notificación (*notify*) o se haya completado el tiempo de espera.

- **TERMINATED**

  Estado que alcanza un hilo cuando finaliza. Un hilo finaliza por cualquiera de las siguientes razones:

  - ➔ Cuando el código del hilo se ha ejecutado completamente, es decir, cuando se ha finalizado la ejecución de todas las sentencias de su método *run()*.
  - ➔ Cuando se produce algún error inusual que provoca una excepción no controlada.

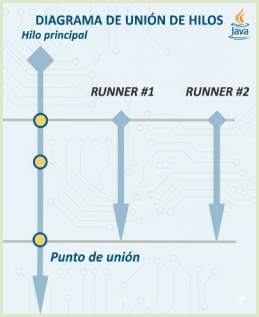
Se usará el método *getState()* para, en cualquier momento, consultar el estado en el que se encuentra un hilo.

Paquete estados: EstadosHilo: Programa cuyo hilo principal lanza un hilo secundario que realiza una cuenta atrás desde 10 hasta 1. Desde el hilo principal se verificará la muerte del hilo secundario mediante la función isAlive(). Además mediante el método getState() de la clase Thread vamos obteniendo el estado del hilo secundario. Se usa también el método join() que espera hasta que el hilo termina.

7. Métodos de la clase java.lang.Thread

Veamos algunos de los métodos de la clase Thread más utilizados:

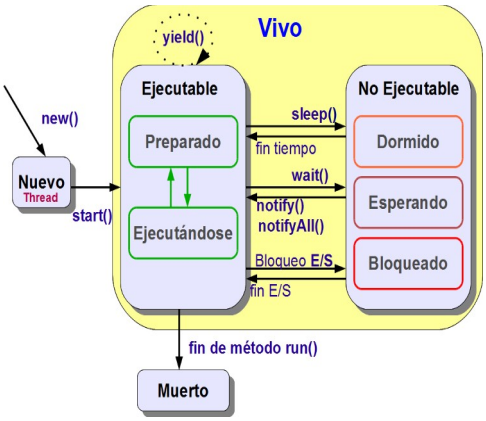
Método	Descripción
boolean isAlive()	Comprueba si un thread está vivo o no. Un hilo se considera que está vivo desde que se llama a su método start() hasta que finaliza su ejecución. Por tanto, este método devolverá: • **False**: Estamos ante un nuevo hilo recién "creado" o ante un hilo "muerto". • **True**: sabemos que el hilo se encuentra en un estado diferente a ***NEW*** y a ***TERMINATED***.
sleep(long ms) sleep (long milisegundos, int nanosegundos)	Cambia el estado del thread a ***TIMED_WAITING*** durante los ms indicados.
String toString()	Devuelve una representación legible de un thread: nombre, priority, nombre_del_grupo.

void join()	Se llama desde otro hilo y hace que el hilo que lo invoca se bloquee hasta que el thread termine. Es parecido a p.waitFor() para los procesos. 

7.1. Detener temporalmente un hilo

¿Qué significa que un hilo se ha detenido temporalmente? Significa que **el hilo ha pasado a un estado "No Ejecutable"**, concretamente al estado ***TIMED_WAITING*** o ***WAITING***.

Un hilo pasará al estado ***TIMED_WAITING (hilo dormido)*** por alguna de estas circunstancias:



- Se ha invocado al método sleep() de la clase Thread**,** indicando el tiempo que el hilo permanecerá deteniendo**.** Transcurrido ese tiempo, el hilo se vuelve "Ejecutable", en concreto pasa al estado ***RUNNABLE***.

  Hay dos formas de llamar al método sleep():

  - La primera le pasa como argumento un entero (positivo) que representa el tiempo en milisegundos que el hilo permanecerá dormido. Se puede usar como parámetro la clase Duration para indicar la duración.
  - La segunda le agrega un segundo argumento entero (esta vez, entre 1 y 999999), que representa un tiempo extra en nanosegundos (Un nanosegundo es la milmillonésima parte de un segundo, (10-9 s)) que se sumará al primer argumento:

- Cualquier llamada a sleep() puede provocar una excepción, que el compilador de Java nos obliga a controlar ineludiblemente mediante un bloque try-catch. Concretamente la excepción a tratar es InterruptedException.

Un hilo pasará al estado ***WAITING (hilo esperando)*** cuando:

- El hilo ha detenido su ejecución mediante la llamada al método wait(), y no se reanudará, no pasará a ***RUNNABLE***, hasta que otro hilo produzca una llamada al método notify() o notifyAll().

  Se estudiará detalladamente estos métodos de la clase Object cuando veamos la sincronización y comunicación de hilos.

Un hilo pasará al estado BLOCKED (hilo bloqueado) cuando:

- El hilo está pendiente de que finalice una operación de E/S en algún dispositivo, o a la espera de algún otro tipo de recurso(ha sido bloqueado por el sistema operativo). Cuando finaliza el bloqueo, vuelve al estado ***RUNNABLE***.

Paquete estados: JoinSleep; Viaje

7.2. Hilos demonio (Daemon Thread)

Todos los hilos vistos hasta ahora son llamados hilos de usuario (**User Threads**). Sin embargo, también existen los conocidos como hilos de sistema (**System Threads**) que la JVM crea automáticamente para manejar tareas internas como gestionar recursos, gestionar señales del sistema operativo, finalización de objetos, etc.

Paquete estados: HilosJVM

Además, en una aplicación se pueden tener hilos demonio (**Daemon Thread**) que sí los puede crear el desarrollador de la aplicación.

Un **Daemon Thread** es un hilo background cuyo propósito es asistir a otros hilos (por ejemplo, liberación de recursos, tareas de limpieza, timers, logging en segundo plano).

La característica esencial de estos hilos es que la JVM los puede finalizar automáticamente si no queda ningún **User Thread** vivo.

Los **Daemon Thread** se crean a partir de hilos de usuario, siendo convertidos en hilos daemon usando el método setDaemon(true). Esto debe hacerse antes de llamar al método start() del hilo. Si se intenta después de la llamada al método start() se lanzará un IllegalThreadStateException.

Por otro lado, para saber si un hilo es un hilo de usuario o un hilo demonio se puede utilizar el método isDaemon().

Paquete daemonthreads: UserThread; DaemonThread

8. Concurrencia moderna en Java. Pool de hilos.

Antes de Java 5, la única forma de gestionar la concurrencia era creando hilos manualmente (new Thread()) y coordinándolos con synchronized, wait() y notify().

Sin embargo, el uso de Thread Pools (o el marco ExecutorService) es el estándar industrial hoy en día.

¿Por qué se debe preferir los pools de hebras sobre la gestión manual?

1. Rendimiento: El costo de crear vs. Reutilizar

Crear un hilo en Java no es gratis, es una operación costosa para el Sistema Operativo.

- **Gestión Manual (new Thread())**: Cada vez que creas un hilo, el sistema debe asignar memoria para la pila (stack) del hilo y realizar llamadas al sistema operativo. Si tu aplicación recibe miles de peticiones, crear y destruir miles de hilos consume muchísima CPU y memoria solo en "burocracia" administrativa.
- **Thread Pool:** Funciona bajo el principio de **reciclaje**. Las hebras se crean una vez y se mantienen vivas. Cuando una tarea termina, la hebra no muere; vuelve al "pool" a esperar la siguiente tarea. Esto elimina la latencia de creación.

2. Estabilidad: Control de Recursos

Este es quizás el punto más crítico para aplicaciones en producción.



- **Gestión Manual**: Si creas un hilo por cada tarea (modelo *unbounded*), un pico de tráfico puede hacer que tu aplicación cree 10,000 hilos simultáneamente. Esto provocará un **OutOfMemoryError** o saturará la CPU con cambios de contexto (context switching), bloqueando el servidor por completo.
- **Thread Pool**: Te permite poner un **límite**. Puedes configurar un pool con un máximo de 50 hebras. Si llegan 100 tareas, 50 se ejecutan y las otras 50 esperan ordenadamente en una cola (**BlockingQueue**). El servidor no se cae, simplemente se mantiene ocupado a su máxima capacidad segura.

La solución a esto pasa por **reusar Threads** para lograr que cada Thread pueda realizar muchas tareas y no solo una tarea y ser destruida.



Para lograrlo se establece un límite de Threads en la aplicación y algún mecanismo que permita asignar diferentes tareas a un mismo Thread.

Por tanto, básicamente lo que se debe decir es "aquí hay un trabajo", y el mecanismo decide qué hilo lo ejecuta, cuándo y cómo. Esto hace el código mucho más limpio y mantenible.



En la imagen se puede ver el funcionamiento. Los Threads esperan a que se le asignen tareas existentes en la cola. Una vez asignada la tarea, el thread correspondiente la ejecuta.

Para esto Java nos ofrece los Executors y los ThreadPoolExecutors. El primero de ellos simplifica la creación de pools sin necesidad de configurar todos sus parámetros manualmente. En cambio, el segundo es mucho más flexible ya que puedes configurar prácticamente todo, por ejemplo:

- número mínimo y máximo de hilos.
- tamaño de la cola de tareas.
- política de rechazo (RejectedExecutionHandler).
- tiempo de espera de hilos inactivos.
- tipo de cola (LinkedBlockingQueue, SynchronousQueue, etc.).

De hecho, los Executors usan internamente los ThreadPoolExecutors. En sistemas críticos de alta carga, no se usa Executors. Se instancia ThreadPoolExecutor manualmente para limitar el tamaño de la cola de espera.

Por tanto, ambos proporcionan un marco para crear y administrar hilos facilitando la labor de:

- **Creación de hilos**: Proporciona varios métodos para crear grupos de hilos (Thread Pools) que la aplicación puede usar para ejecutar tareas al mismo tiempo.
- **Gestión de hilos**: No hay que preocuparse por si los hilos están activos, ocupados o destruidos antes de enviar una tarea para su ejecución. No es necesario hacer llamadas a los métodos start() ni join() ya que la propia herramienta se encarga de la creación, ejecución y parada de la hebra.
- **Envío y ejecución de tareas**: Proporcionan métodos para enviar tareas para su ejecución y para decidir cuándo se ejecutarán dichas tareas (programar las tareas o ejecución periódica de tareas).
- **Devolución de datos: Permite que las tareas asignadas a las hebras puedan devolver datos.**

8.1. Executors

Java a través de su API java.util.concurrent proporciona varias interfaces basadas en una interfaz base (Executors) que es una factoría que consta de varios métodos estáticos para la creación de diferentes tipos de Pools.





Los métodos estáticos de Executors son los siguientes:

Método	Descripción	Uso
newFixedThreadPool (númeroHilos)	Crea un Pool con un número fijo de hilos. Consta de una cola ilimitada para tareas pendientes. Una vez que todos los hilos han sido creados y están ocupados, si llegan nuevas tareas, estas se irán a la cola de tareas a esperar que algún hilo quede disponible. Cuando una tarea finaliza, el hilo queda disponible para que se le asigne una nueva tarea.	Cuando conoces la carga de trabajo y quieres evitar saturar la CPU. Por ejemplo: • Servidores web para manejar request HTTP con carga predecible. • Procesamiento de archivos o imágenes. • APIs REST para procesar llamadas concurrentes. • Procesamiento de registros de bases de datos.
newSingleThreadExecutor()	El pool consta de un solo hilo. Por lo tanto, solo se puede ejecutar una tarea al mismo tiempo. Si el hilo se destruye por alguna circunstancia, vuelve a ser creado.	Adecuado cuando se tienen tareas secuenciales.
NewCachedThreadPool (númeroHilos)	Igual que el primero con la salvedad de que si un hilo está unos 60 segundos sin ser usado, automáticamente será eliminado. Por tanto, los threads se van creando según son demandados. En este caso no hay colas de tareas, se van creando threads adicionales si todos los creados hasta ese momento están ocupados.	Se suelen usar para: • Microservicios • Tareas cortas y asíncronas.
newScheduledThreadPool(n úmeroHilos)	Permite programar la ejecución de las tareas. Tiene el inconveniente de que si el tamaño del Pool es pequeño y las tareas tienen larga duración, las nuevas tareas pueden retrasarse.	Usarlo cuando sean necesario ejecutar tareas en un instante de tiempo o en intervalos regulares, teniendo siempre presente la posibilidad de retraso. Ejemplos de uso: • Backups automáticos. • Reportes periódicos, como por ejemplo, generar estadísticas cada hora. • Health Checks: Verificación de servicios cada cierto tiempo. • Limpieza de caché: Eliminación de datos expirados.

Por defecto, la aplicación que usa un pool de hilos no finalizará a pesar de que los hilos no estén ejecutando ninguna tarea porque estas hayan ya terminado. Para evitar esto Java proporciona varios métodos:

- shutdown(): Deja de aceptar nuevas tareas. Espera a que las tareas que ya están corriendo y las que están en la cola terminen. En ese momento ExecutorService se cerrará.

- shutdownNow(): Intenta detener las tareas activas (envía una interrupción Thread.interrupt()) y devuelve una lista de las tareas que estaban en cola y no se llegaron a ejecutar. En este caso no hay garantías de que las tareas en ejecución se detengan o realicen su ejecución hasta el final, va a depender del código que se tenga implementado en el tratamiento de la interrupción (código del catch).

- awaitTermination(long tiempo, TimeUnit unidad): Bloquea el hilo de ejecución principal hasta que ExecutorService se cierre por completo o hasta que se consuma el tiempo de espera indicado. Este método suele ser llamado después de cualquiera de los dos anteriores. Devuelve **true** si el ejecutor terminó o **false** si transcurrió el tiempo sin que haya terminado.

Otros métodos de la clase Executors son:

- isShutdown() indica que se ha solicitado el apagado.

- isTerminated() indica que el Executor ya ha terminado.

Un executor puede tener isShutdown() == true e isTerminated() == false mientras sus tareas siguen trabajando.

Aspecto	shutdown()	shutdownNow()
Nuevas tareas	Deja de aceptarlas.	Deja de aceptarlas.
Tareas en ejecución	Las deja continuar sin solicitar su interrupción.	Intenta interrumpirlas.
Tareas esperando en cola	Se ejecutarán cuando haya hilos disponibles.	Se retiran de la cola y no se ejecutan.
Valor devuelto	Ninguno: **void**.	Una **List<Runnable>** con las tareas retiradas de la cola.
¿Espera a que termine el executor?	No.	No.

8.2. ExecutorService

Subinterfaz de Executor que a través de los métodos execute(Runnable) o submit(Runnable) ejecuta la tarea pasada como parámetro cuando le sea posible en uno de los hilos del Pool.

Por tanto, a estos métodos hay que llamarlo una vez por cada tarea que deba ser ejecutada por el pool.

El ExecutorService sería como un gerente de recursos humanos al que le damos trabajos (la tarea) y él decide qué trabajador (hilo) la ejecuta y cuándo.

Proyecto Concurrencia Moderna - Paquete executor: SingleExecutor; FixedExecutor; EjemploShutdownNow; EjemploAwaitTermination; ApagadoEnDosFases



8.3. ScheduledExecutorService

Subinterfaz de ExecutorService que proporciona funcionalidades para programar la ejecución de tareas.

Respecto a la planificación de tareas Java ofrece tres métodos de planificación de tareas, permitiendo dos de ellos la repetición de tareas:

- schedule(Runnable, delay, unit)

  Uso: "Ejecuta esto UNA sola vez dentro de xx unidades de tiempo". No es periódico. Es un disparo retardado.

- scheduleAtFixedRate(Runnable, initial, period, unit)

  Intenta mantener una frecuencia estricta. El reloj empieza a contar desde el inicio de la ejecución anterior.
  Ejemplo: Si configuras cada 10s, las tareas se lanzarán a las 12:00:00, 12:00:10, 12:00:20…

  Riesgo: Si tu tarea tarda 15s en ejecutarse y el periodo es de 10s, las tareas se empezarán a solapar o amontonar en la cola, consumiendo recursos rápidamente.

- scheduleWithFixedDelay(Runnable, initial, delay, unit)

  Lógica: Garantiza un "descanso". El reloj empieza a contar desde el final de la ejecución anterior.

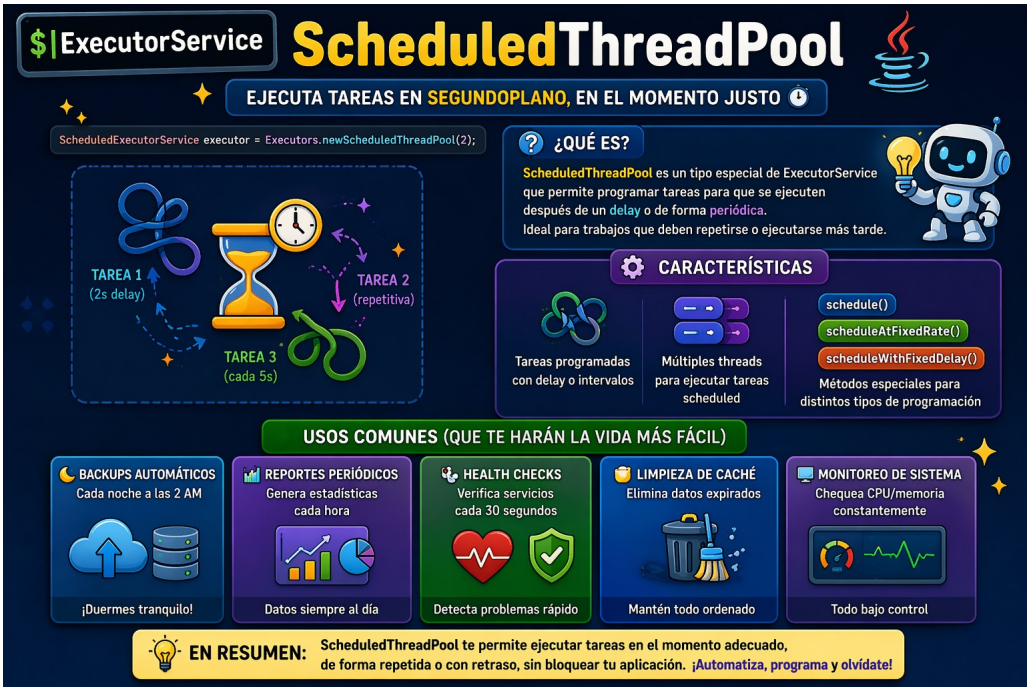
  Ejemplo: Tarea tarda 5s, descanso (delay) de 10s. Inicio: 00s -> Fin: 05s -> (espera 10s) -> Inicio: 15s.

  Ventaja: Es más seguro para tareas pesadas o de duración variable, ya que garantiza que el sistema respire.

Por tanto, la diferencia entre estos dos últimos estriba en que el FixedRate ignora cuánto tarda la tarea (mientras sea menor al periodo) y dispara rigurosamente a :00, :03, :06. Mientras que el FixedDelay espera a terminar (:03) y suma el delay (4s), lanzando el siguiente a las :07.

Proyecto ConcurrenciaModerna - Paquete executor: ScheduledExecutor;

ScheduledRateDelayExecutor



9. Callable y Future

Las formas previamente explicadas sobre la creación de hebras se caracterizan por no devolver un resultado cuando el método run() finaliza. Para este tipo de situaciones se usará la interfaz Callable, es decir, se usará para definir tareas que devuelven un resultado.

Callable al igual que Runnable es una interfaz funcional usada para representar tareas que pueden ser ejecutadas por Threads o por ExecutorService, aunque hay algunas diferencias entre ambas interfaces:

- Runnable: Representa una tarea que no devuelve un resultado y no puede lanzar una excepción comprobada. La tarea asociada puede ser ejecutada usando los métodos execute() o submit() de un objeto ExecutorService. Existente desde Java 1.0.

- Callable: Representa una tarea que devuelve un resultado de tipo Future y puede lanzar una excepción comprobada. Se debe ejecutar usando el método submit de un objeto ExecutorService. Existente desde Java 5.0.




```
Callable<String> tarea=()->{ return "Hola desde Callable";};
ExecutorService executor=Executors.newSingleThreadExecutor();
Future<String> future=executor.submit(tarea);
String result=future.get(); //Se bloquea hasta que el resultado esté disponible.
System.out.println(result);
```

Proyecto ConcurrenciaModerna - Paquete future.callable: TareasCallable;

Al igual que ocurre con la interfaz Runnable, con la interfaz Callable, no solo se pueden crear tareas usando expresiones lambda (tal y como se ha visto en los dos ejemplos anteriores). También se pueden crear clases que implementen la interfaz Callable pero en lugar de implementar un método run() debe implementarse un método call(). En este caso debe indicarse el tipo de datos a devolver tanto en la cabecera de la clase como en la cabecera del método call().

```
public class HiloMaximo implements Callable<Integer>
{
	private int [] numeros;
	private int inicio, fin;
	private String nombre;

	….
	@Override
	 public Integer call() {
	{
		int maximo;
		….
		return maximo;
	}
}
```

Además, el método call() también puede lanzar excepciones que no queremos que sean tratadas por él. En tal caso hay que hacer el correspondiente throws en la cabecera del método.

```
public class ProcesaPedido implements Callable<Integer>
{
	private final int numeroPedido;
	private final int unidades;
	private final double precioUnitario;

	….
	@Override
	 public Integer call() throws InterruptedException {
	{
		//Simulamos el tiempo que tarda en procesar el pedido con un sleep.
		Thread.sleep(1000);
		….
		return unidades*precioUnitario;
	}
}
```

Proyecto ConcurrenciaModerna - Paquete future.callable: MaximoArray; ProcesamientoPedidos

9.1 Problemas de Future y ExecutorService

El uso de Future y ExecutorService en Java fue durante muchos años el modelo estándar para manejar concurrencia, pero tienen varias limitaciones importantes que motivaron la aparición de CompletableFuture y posteriormente Virtual Threads. Además, Java también ha evolucionado hacia la Programación Reactiva (construir aplicaciones que procesan eventos de forma asíncrona) y hacia la concurrencia estructurada.

Inconvenientes del uso de Future en Java

- **Bloqueo al obtener el resultado (get() es bloqueante)**

  El método future.get(); bloquea el hilo hasta que la tarea termina. Esto rompe uno de los objetivos principales de la programación concurrente: evitar bloqueos innecesarios.

```
Future<String> future = executor.submit(task);
String resultado = future.get(); // BLOQUEA
```

  Si la tarea tarda 5 segundos, 30 segundos o nunca termina, el hilo queda bloqueado.

  A veces para evitar el bloqueo indefinido, se usa una solución no ideal porque sigue siendo bloqueante aunque por tiempo definido:

```
String resultado=future.get(5, TimeUnit.SECONDS); //bloquea durante 5 segundos
```

  En este caso, si transcurre los 5 segundos y no se ha obtenido el resultado de la tarea, saltará una excepción de tipo TimeoutException que deberá ser tratada. Por tanto, si se usa un bloqueo por tiempo limitado deberá organizarse el código del siguiente modo:

```
try {
	String resultado = future.get(5, TimeUnit.SECONDS);
	// Solo llegamos aquí si hemos obtenido correctamente el resultado.
	procesaResultado(resultado);
} catch (TimeoutException e) {
	System.out.println("Han pasado cinco segundos y todavía no tenemos el resultado.");
	System.out.println("Dado que es un dato que necesito procesar me voy a esperar para obtenerlo.");
	String result = future.get();
	procesaResultado(result);
} catch (ExecutionException e) {
	System.out.println("La tarea ha fallado: " + e.getCause());
	System.out.println(e.getMessage());
} catch (InterruptedException e) {
	Thread.currentThread().interrupt();
	System.out.println("El hilo principal ha sido interrumpido.");
} catch (CancellationException e) {
	System.out.println("La tarea ha sido cancelada.");
}
```

Cabe destacar que la tarea puede seguir ejecutándose: **el agotamiento del plazo no la cancela**.

Si queremos obtener el resultado se debe llamar otra vez a get() sobre el mismo Future, no siendo necesario volver a enviar la tarea. Esto se podría hacer en el TimeoutException.

Un aspecto con el que se debe tener mucho cuidado es con poner un mismo try ...catch(TimeoutException) para varios get(tiempo,TimeUnit).

```
try {
    String resultado1 = future1.get(5, TimeUnit.SECONDS);
    Integer resultado2 = future2.get(5, TimeUnit.SECONDS);
    Double resultado3 = future3.get(5, TimeUnit.SECONDS);

} catch (TimeoutException e) {
    // Si falla la primera espera, no se ejecutan los otros dos get.
}
```

En este caso, si transcurren los 5 segundos del primer get(), el código salta al TimeoutException y por tanto, se quedará sin obtener el resultado del future2 y del future3.

Lo que se debe hacer es no seguir esperando ahora por la tarea que ha superado el tiempo de espera y seguir atendiendo a las demás. Lo que sí se debe es decidir que hacer con esas tareas que han quedado pendientes:

- Volver a consultarlos más adelante. Esta solución requerirá tener una lista de los Future para recorrerla con los get() que han quedado pendientes.
- Solicitar su cancelación.
- Informar de que no se obtuvo el resultado dentro del plazo y continuar sin él.

Proyecto ConcurrenciaModerna - Paquete future.callable: TimeOut

- **No permite encadenar tareas sin bloqueos (no hay composición)**

La interfaz Future no ofrece métodos para registrar acciones que se activen al completarse una tarea. Podemos coordinar tareas dependientes manualmente mediante get(), bloqueando al hilo coordinador mientras espera.

Future no permite, **sin bloqueos**, que teniendo tres tareas A, B y C, hacer algo como:

cuando termine A -> ejecutar B

cuando termine B -> ejecutar C

```
Future<User> userFuture = executor.submit(getUser());

Future<Address> addressFuture = executor.submit(getAddress(userFuture.get()));
```

equivalen a:

```
// Enviamos la primera tarea.
Future<User> userFuture = executor.submit(getUser());

// El hilo que ejecuta estas líneas espera al usuario.
User user = userFuture.get();

// Cuando lo tenemos, construimos la segunda tarea.
Callable<Address> tareaDireccion = getAddress(user);

// Finalmente, enviamos la segunda tarea.
Future<Address> addressFuture = executor.submit(tareaDireccion);
```

- **Manejo de errores limitado y coordinación de tareas compleja**

```
Future<String> cliente = executor.submit(() -> consultarCliente());

Future<Double> importe = executor.submit(() -> calcularImporte());

String nombreCliente = cliente.get();
double total = importe.get();
```

Las tareas se ejecutan concurrentemente, pero estos dos Future no establecen por sí mismos una política conjunta.

Por ejemplo:

- Si falla la consulta del cliente, el cálculo del importe puede seguir ejecutándose.
- Si el importe falla inmediatamente, pero estamos esperando en cliente.get(), podemos tardar en observar ese fallo.
- Tenemos que programar cómo cancelar las tareas que ya no necesitamos.

- **Cancelación no garantizable**

  Cancelar el Future no garantiza que el trabajo haya dejado de ejecutarse. Además, coordinar la cancelación de varias tareas relacionadas requiere trabajo adicional por parte del programador necesita coordinación explícita.